SQL

6

SQL *("ess-que-el" olarak telaffuz edilir)* **Yapısal Sorgu Dili**'nin kısaltmasıdır. SQL bir veritabanı ile iletişim kurmak için kullanılır. **ANSI** *(Amerikan Ulusal Standartlar Enstitüsü)*'ye göre, ilişkisel veritabanı yönetim sistemleri için standart dildir. SQL deyimleri, bir veritabanındaki verileri güncellemek veya bir veritabanından veri almak gibi görevleri yerine getirmek için kullanılır. SQL'i kullanan bazı ortak veritabanı veritabanı yönetim sistemleri şunlardır: Oracle, Sybase, Microsoft SQL Server, Access, vs. Çoğu veritabanı sistemi SQL kullansa da çoğu sistemin kendi ek tescilli uzantıları vardır, bunlar genellikle yalnızca kendi sistemlerinde kullanılır.

NOT MySQL kurulumu ile gelen (eğer silmemişseniz) "sakila" veritabanını kullanacağız.

SELECT (Seçme)

Syntax:

```
SELECT
[ALL|DISTINCT|DISTINCTROW]
[HIGH_PRIORITY]
[STRAIGHT_JOIN]
[SQL_SMALL_RESULT][SQL_BIG_RESULT][SQL_BUFFER_RESULT]
[SQL_CACHE|SQL_NO_CACHE][SQL_CALC_FOUND_ROWS]
select_expr[,select_expr...]
[FROMtable_references
```

```
[PARTITIONpartition_list]
[WHEREwhere_condition]
[GROUPBY {col_name|expr|position}
[ASC|DESC],...[WITHROLLUP]]
[HAVINGwhere_condition]
[ORDERBY {col_name|expr|position}
[ASC|DESC],...]
[LIMIT {[offset,]row_count|row_countOFFSEToffset}]
[PROCEDUREprocedure_name(argument_list)]
[INTOOUTFILE'file_name'
[CHARACTERSETcharset_name]
export_options
|INTODUMPFILE'file_name'
|INTOvar_name[,var_name]]
[FORUPDATE|LOCKINSHAREMODE]]
```

SELECT yapısını yukardaki syntax gövdesini görmekteyiz. SELECT yapısını bir döngü elemanı gibi kullanabilirsiniz.

SELECT Yapısı ile Matematiksel İşlemler:

```
Mysql> SELECT 1+1;
```

Çıktı: 2

```
Mysql> SELECT TRUE+TRUE;
```

Çıktı: 2

Burada **TRUE** değeri **1** olarak görülmektedir.

```
Mysql> SELECT FALSE+TRUE;
```

Çıktı: 1

FALSE değeri **0** ile temsil ediliyor. 0 değerinin 0 ile bölünmesi (0/0) tanımsız olacağından sorgudan çıktı NULL olacaktır.

Bir MySQL fonksiyonu olan IFNULL();

```
Mysql> SELECT IFNULL((0/0),5);
```

Çıktı: 5

Burada matematiksel tüm işlemleri yapabilirsiniz. Bu örnekler de temel olarak **SELECT** davranışını görmekteyiz. Aynı şekilde bir tabloadaki alanları birbirleriyle değer olarak matematiksel işlemlerinde kullanılabilinir. Örneğimizdeki (+) OR, (*) ise AND temsil etmektedir. "**address**" tablosundaki tüm kayıtları ve tüm alanları ile döndürmek istersek;

Query Builder | Query Editor

```
1  SELECT * FROM address
```

Message | Result1 | Profile | Status

address_id	address	address2	district	city_id	postal_code
1	47 MySakila Drive	(Null)	Alberta	300	
2	28 MySQL Boulevard	(Null)	QLD	576	
3	23 Workhaven Lane	(Null)	Alberta	300	
4	1411 Lillydale Drive	(Null)	QLD	576	
5	1913 Hanoi Way		Nagasaki	463	35200
6	1121 Loja Avenue		California	449	17886
7	692 Joliet Street		Attika	38	83579

Basit Bir SELECT Sorgusu

"*" **yıldız** karekteri tüm alanları döndürmek için kullanılır. Sadece belirli alanları döndürmek istersek;

```
1  SELECT address,district FROM address
```

Message | Result1 | Profile | Status

address	district
47 MySakila Drive	Alberta
28 MySQL Boulevard	QLD
23 Workhaven Lane	Alberta
1411 Lillydale Drive	QLD
1913 Hanoi Way	Nagasaki

Sadece Belli Sütünların Görülmesi İsteniyorsa

Örneklerimizde alan ve tablo isimlerini olduğu gibi sorguda kullanıldı. SQL'de **alias** *(as)* ile döndürülen alan ve tablo adlarını değiştirebiliriz.

Query Builder | Query Editor

```
1  SELECT address AS adr,district AS d,ornek(city_id) AS c FROM address AS a
```

Message | Result1 | Profile | Status

adr	d	c
47 MySakila Drive	Alberta	Lethbridge
28 MySQL Boulevard	QLD	Woodridge
23 Workhaven Lane	Alberta	Lethbridge
1411 Lillydale Drive	QLD	Woodridge
1913 Hanoi Way	Nagasaki	Sasebo

Sütünları Alias Kullanılarak Gösterim

Fonksiyon ve sabitleri kullanabiliriz.

File Edit Format View Window Help

Run | Stop | Explain | Export Wizard | New | Load | Save | Save As | Find | Word Wrap | Grid View

Query Builder | Query Editor

```
1  SELECT VERSION(),USER(),DATABASE(),TRUE,FALSE,NULL;
```

Message | Result1 | Profile | Status

VERSION()	USER()	DATABASE()	TRUE	FALSE	NULL
5.7.20	root@localhost	sakila	1	0	(Null)

MySQL Yerleşik Fonksiyonların Kullanılması

Tekrarlayan alan değerlerini tekrarlanmasını durdurmak için *(DISTINCT)*;

Query Builder | Query Editor

```
1  SELECT DISTINCT city_id from address
```

Message | Result1 | Profile | Status

city_id
1
2
3
4
5
6
7
8

DISTINCT Kullanımı

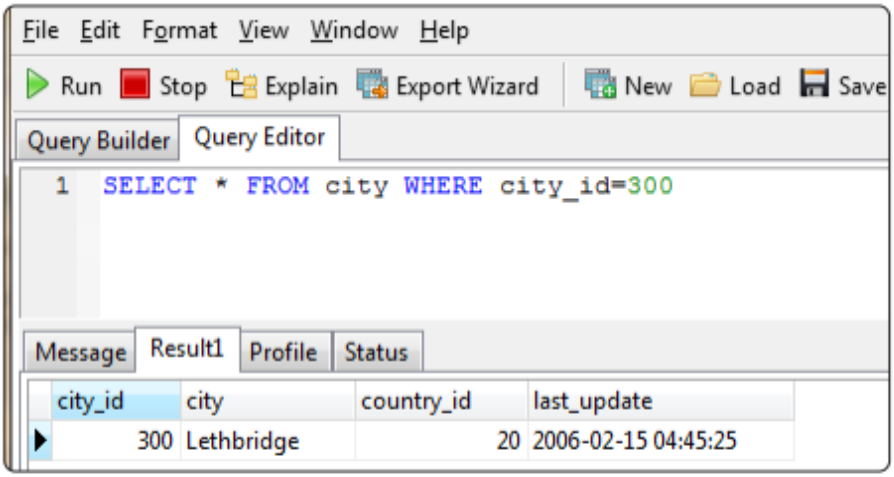
WHERE Filtreleme Cümleciği

WHERE, seçim yaparken (**SELECT**) kullandığımız gibi güncelleme (**UPDATE**) ve silme (**DELETE**) için de kullanırız.

Söz Dizimi(Syntax):

```
SELECT sutun1, sutun2, ...
FROM tablo_ismi
WHERE şartlar;
```

En basit haliyle bir WHERE cümleciğinin kullanılması;



WHERE Kullanımı

Where Cümleciğinin Operatörleri:

Operator	Description
=	Eşit
<>	Eşit değil
>	Büyüktür
<	Küçüktür
>=	Eşit ve Büyük
<=	Eşit ve Küçük
BETWEEN	Arasında
LIKE	String Arama
IN	IN() değerler içinde

BETWEEN Operatörü: BETWEEN operatörü, belirli bir aralıktaki değerleri seçer. Değerler sayı, metin veya tarih olabilir.

BETWEEN operatörü kapsayıcıdır: başlangıç ve bitiş değerleri dahildir.

Query Builder | Query Editor

```
1  SELECT * from actor WHERE actor_id BETWEEN 5 AND 10
```

Message | Result1 | Profile | Status

actor_id	first_name	last_name	last_update
5	JOHNNY	LOLLOBRIGIDA	2006-02-15 04:34
6	BETTE	NICHOLSON	2006-02-15 04:34
7	GRACE	MOSTEL	2006-02-15 04:34
8	MATTHEW	JOHANSSON	2006-02-15 04:34
9	JOE	SWANK	2006-02-15 04:34
10	CHRISTIAN	GABLE	2006-02-15 04:34

WHERE Operatörlerle Kullanımı

IN Operatörü: Değer vererek kayıtları çekmek istersek;

Run | Stop | Explain | Export Wizard | New | Load | Save | Save As | Find

Query Builder | Query Editor

```
1  SELECT * FROM address WHERE city_id IN(300,200)
```

Message | Result1 | Profile | Status

address_id	address	address2	district	city_id	postal_code	phon
12	478 Joliet Way		Hamilton	200	77948	65728
1	47 MySakila Drive	(Null)	Alberta	300		
3	23 Workhaven Lane	(Null)	Alberta	300		14033

IN Operatörünün Kullanımı

Alt sorgularla kayıt seçmek istersek;

File Edit Format View Window Help

Run Stop Explain Export Wizard New Load Save Save As Find Word Wrap

Query Builder Query Editor

```
1  SELECT * FROM address WHERE city_id IN(SELECT city_id FROM city)
```

Message Result1 Profile Status

address_id	address	address2	district	city_id	postal_code	phone	location
222	1168 Najafabad Parkway		Kabol	251	40301	886649065861	(BLOB)
446	1924 Shimonoseki Drive		Batna	59	52625	406784385440	(BLOB)
73	1031 Daugavpils Parkway		Bchar	63	59025	107137400143	(BLOB)
180	757 Rustenburg Avenue		Skikda	483	89668	506134035434	(BLOB)
325	1892 Nabereznyje Telny I		Tutuila	516	28396	478229987054	(BLOB)
534	486 Ondo Parkway		Benguela	67	35202	105882218332	(BLOB)
388	368 Hunuco Boulevard		Namibe	360	17165	106439158941	(BLOB)
386	1368 Maracabo Boulevar			493	32716	934352415130	(BLOB)

IN Operatörünün Alt Sorgu ile Kullanımı

Like Operatörü: a harfi ile başlayan "**city**" kayıtlarını getir.

Query Builder Query Editor

```
1  SELECT * FROM city WHERE city LIKE 'a%'
```

Message Result1 Profile Status

city_id	city	country_id	last_update
1	A Corua (La Corua)	87	2006-02-15 04:45:25
2	Abha	82	2006-02-15 04:45:25
3	Abu Dhabi	101	2006-02-15 04:45:25
4	Acua	60	2006-02-15 04:45:25
5	Adana	97	2006-02-15 04:45:25

Like Operatörünün Kullanımı

Like '%a' : a ile biten;

Like '%dan%' : içerisinde 'dan' bulunan kayıtlar.

Query Builder | Query Editor

```
1  SELECT * FROM city WHERE city LIKE '%dan%'
```

Message | Result1 | Profile | Status

city_id	city	country_id	last_update
5	Adana	97	2006-02-15 04:45:25
327	Mardan	72	2006-02-15 04:45:25

Like Operatörünün Kullanımı

Like '_r%': İkinci harfi r olan kayıtları döndürür.

Query Builder | Query Editor

```
1  SELECT * FROM city WHERE city LIKE  '_r%'
```

Message | Result1 | Profile | Status

city_id	city	country_id	last_update
30	Araatuba	15	2006-02-15 04:45:25
31	Arak	46	2006-02-15 04:45:25
32	Arecibo	77	2006-02-15 04:45:25
33	Arlington	103	2006-02-15 04:45:25
88	Bradford	102	2006-02-15 04:45:25

Like Operatörünün Kullanımı

GROUP BY VE HAVING CÜMLECİĞİ

GROUP BY yan tümcesi, döndürülen kaydı bir veya daha çok sütunla gruplar. Sonuç kümesinin hangi sütunlara göre gruplandığını belirtirsiniz. Aşağıdaki sonuç kümesini göz önünde bulundurun:

Query Builder | Query Editor

```
1  SELECT last_name FROM actor
```

Message | Result1 | Profile | Status

last_name
AKROYD
AKROYD
AKROYD
ALLEN
ALLEN
ALLEN

Basıt SELECT Kullanımı

DESC

ASC tam tersidir. Büyükten küçüğe doğru sıralama yapar. *(Z-A)*

Query Builder | Query Editor

```
1  SELECT * from actor ORDER BY actor_id DESC
```

Message | Result1 | Profile | Status

actor_id	first_name	last_name	last_update
200	THORA	TEMPLE	2006-02-15 04:34
199	JULIA	FAWCETT	2006-02-15 04:34
198	MARY	KEITEL	2006-02-15 04:34
197	REESE	WEST	2006-02-15 04:34
196	BELA	WALKEN	2006-02-15 04:34
195	JAYNE	SILVERSTONE	2006-02-15 04:34
194	MERYL	ALLEN	2006-02-15 04:34
193	BURT	TEMPLE	2006-02-15 04:34
192	JOHN	SUVARI	2006-02-15 04:34

DESC Kullanımı

ALT SORGULAR

Alt sorgular, WHERE yan tümcesindeki başka bir sorgunun sonuçlarını kullanmanıza izin veren güçlü bir kavramdır. Bu, sorgunun sonucuna bağlı olacağı için, WHERE yan tümcesine dinamik bir değer iletmenize izin verir. Bir alt sorgu için bir örnek:

Query Builder | Query Editor

```
1  SELECT * FROM actor
2  WHERE actor_id IN
3    (SELECT actor_id FROM film_actor
4    WHERE film_id = 2);
```

Message | Result1 | Profile | Status

actor_id	first_name	last_name	last_
19	BOB	FAWCETT	2006
85	MINNIE	ZELLWEGER	2006
90	SEAN	GUINESS	2006
160	CHRIS	DEPP	2006

Alt Sorguların Kullanımı

Yukarıdaki örnekte, alt sorgu parantez içine alınmıştır. Filmde görünen tüm aktörlerin ayrıntılarını bir **film_id 2** ile döndürürüz. Bunu yapabiliriz, çünkü **film_actor** tablosu **actor_id** ve **film_id** sütunlarını içerir.

İç İçe Alt Sorgular

Bir alt sorguyu, gerekirse başka bir alt sorgu içerisinde yerleştirebilirsiniz.

Yukarıdaki örnekte, **film_id** öğesini kullanmak zorundaydık çünkü **film_actor** tablosu film başlığını içermiyordu. Bakmamız gereken her film için film kimliğini hatırlamak yerine film başlığını kullanabilirsek daha güzel olur. Burası iç içe geçmiş bir alt sorgu kullanışlı olabilir. Bunun gibi iç içe geçmiş bir alt sorgu kullanabiliriz:

Query Builder | Query Editor

```
2  WHERE actor_id IN
3    (SELECT actor_id FROM film_actor
4    WHERE film_id =
5      (SELECT film_id FROM film
6      WHERE title = 'Ace Goldfinger')
7    );
```

Message | Result1 | Profile | Status

actor_id	first_name	last_name	last_
19	BOB	FAWCETT	2006
85	MINNIE	ZELLWEGER	2006
90	SEAN	GUINESS	2006
160	CHRIS	DEPP	2006

İç İçe Sorgu Örneği

Türetilmiş Tablo (Derived Tables)

MySQL 5.7'de, türetilmiş tabloların ve görünümlerin daha tutarlı bir şekilde işlenebilmesi için optimizer'i geliştirildi.

Tekrar özetlemek gerekirse: Türetilen bir tablo, bir SQL ifadesinin **FROM** yan tümcesindeki bir tablonun yerini alabilecek bir alt sorgudur. Sorguda gömülü türetilmiş bir tablo bazen adsız bir türetilmiş tablo olarak adlandırılır. Buna basitçe "**FROM cümlecinde alt sorgu**" denir. Türetilmiş bir tabloyu bir görünümle değiştirmek de mümkündür. Bu nedenle bazen görüşlerin türetilmiş tablolar olarak adlandırılır. Aşağıdaki örnekte FROM sözcüğünden sonra bir tablo ya da view gelmeyip, sonuç döndüren bir sanal tablo oluşmakta ve bu tablonun alanlarına erişerek sonuç döndürebilmekteyiz.

Türetilmiş bir tablonun ne zaman birleştirileceğinin ölçütleri, görünümler için olduğu gibi, alt sorgu içeriyorsa, birleştirme desteklenmez:

```
1  SELECT AVG(a) FROM
2    (SELECT
3      SUM(amount) a
4    FROM payment
5    GROUP BY customer_id) AS totals;
```

AVG(a)
112.548431

Türetilmiş Tablolar

MySQL Yerleşik Fonksiyonları

Toplam (Küme) Fonksiyonları

AVG: Bir dizi değerin veya bir ifadenin ortalama değerini hesaplar.

```
1  SELECT AVG(amount) FROM payment a
```

AVG(amount)
4,200667

AVG Kullanımı

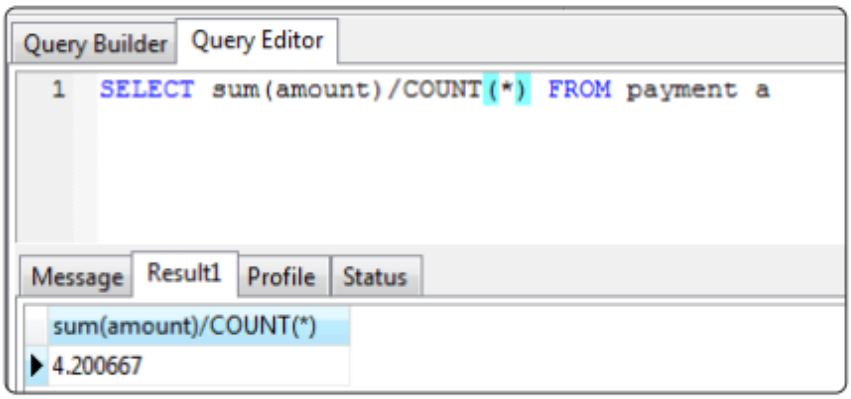
Count: Bir dizideki satır sayılarını getirir.

```
1  SELECT COUNT(amount) FROM payment a
```

COUNT(amount)
16049

COUNT Kullanımı

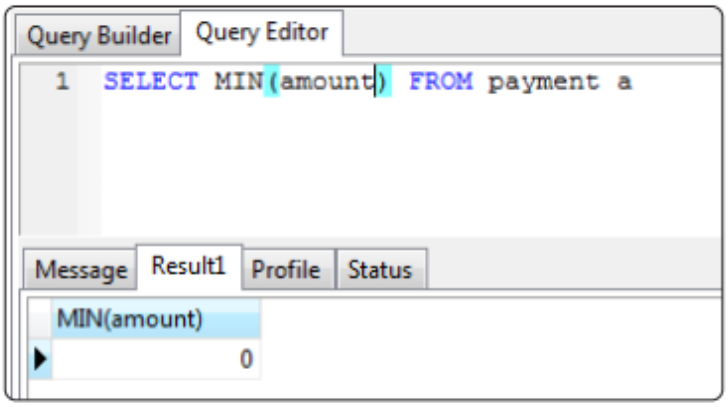
SUM: Bir dizi değer veya bir ifade toplamını hesaplar.



SUM Kullanımı

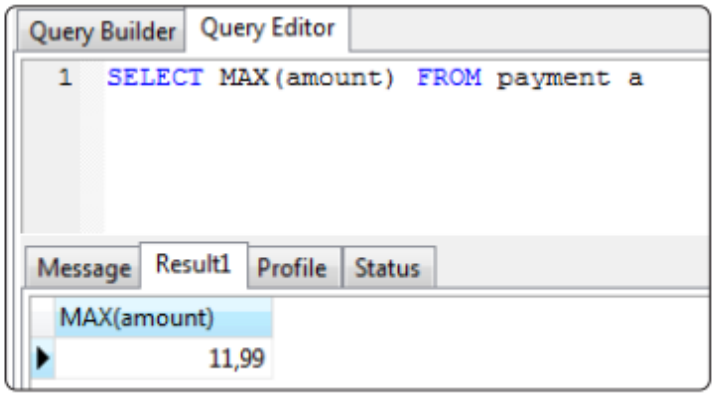
Ortalama değerinin sağlamasını yapmış olduk.

MIN: Bir dizideki en küçük değeri döndürür.



MIN Kullanımı

MAX: Bir dizideki en büyük değeri döndürür.



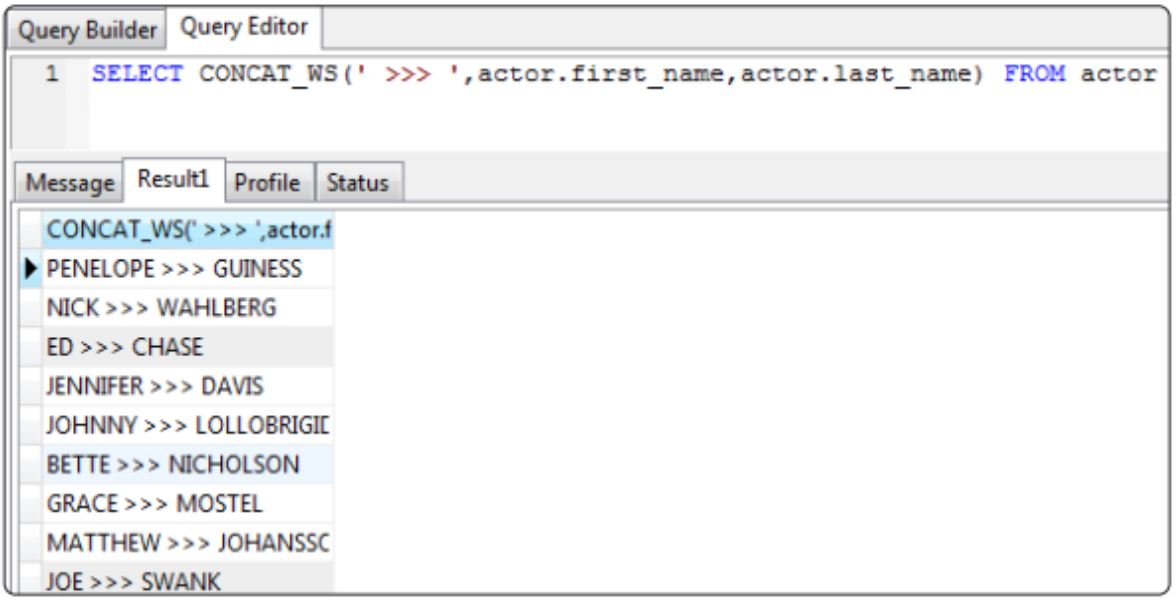
MAX Kullanımı

String Fonksiyonlar

GROUP_CONCAT: Bir dizideki *(tablo, vb)* sütundaki değerleri virgülle ayırarak birleştirir. Bu birleştirme string uzunluğu 1000 ile sınırlıdır. Bu sınırı artırmak için;

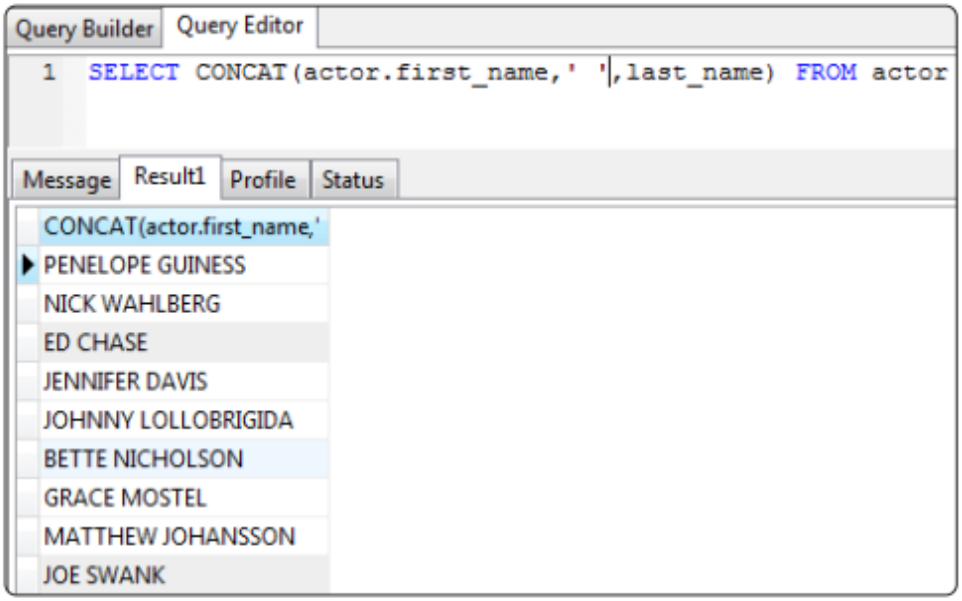
```
Mysql>SET SESSION group_concat_max_len = 1000000;
```

Virgülle değil de kendimizin belirlediği ayırma karakteri ile yapmak istersek;



CONCAT_WS Kullanımı

CONCAT: İki veya daha fazla string değeri birleştirir.



Concat Yerleşik Fonksiyonun Kullanışı

Bu adresten string fonksiyonlarının listesine ulaşabilirsiniz: *https://dev.mysql.com/doc/refman/5.7/en/string-functions.html*